

SECTION 1

INTRODUCTION

This manual contains detailed information about software instructions used by the microprocessors and coprocessors in the M68000 family, including:

MC68000	—	16-/32-Bit Microprocessor
MC68EC000	—	16-/32-Bit Embedded Controller
MC68HC000	—	Low Power 16-/32-Bit Microprocessor
MC68008	—	16-Bit Microprocessor with 8-Bit Data Bus
MC68010	—	16-/32-Bit Virtual Memory Microprocessor
MC68020	—	32-Bit Virtual Memory Microprocessor
MC68EC020	—	32-Bit Embedded Controller
MC68030	—	Second-Generation 32-Bit Enhanced Microprocessor
MC68EC030	—	32-Bit Embedded Controller
MC68040	—	Third-Generation 32-Bit Microprocessor
MC68LC040	—	Third-Generation 32-Bit Microprocessor
MC68EC040	—	32-Bit Embedded Controller
MC68330	—	Integrated CPU32 Processor
MC68340	—	Integrated Processor with DMA
MC68851	—	Paged Memory Management Unit
MC68881	—	Floating-Point Coprocessor
MC68882	—	Enhanced Floating-Point Coprocessor

NOTE

All references to the MC68000, MC68020, and MC68030 include the corresponding embedded controllers, MC68EC000, MC68EC020, and MC68EC030. All references to the MC68040 include the MC68LC040 and MC68EC040. This referencing method applies throughout the manual unless otherwise specified.

The M68000 family programming model consists of two register groups: user and supervisor. User programs executing in the user mode only use the registers in the user group. System software executing in the supervisor mode can access all registers and uses the control registers in the supervisor group to perform supervisor functions. The following paragraphs provide a brief description of the registers in the user and supervisor models as well as the data organization in the registers.

1.1 INTEGER UNIT USER PROGRAMMING MODEL

Figure 1-1 illustrates the integer portion of the user programming model. It consists of the following registers:

- 16 General-Purpose 32-Bit Registers (D7 – D0, A7 – A0)
- 32-Bit Program Counter (PC)
- 8-Bit Condition Code Register (CCR)

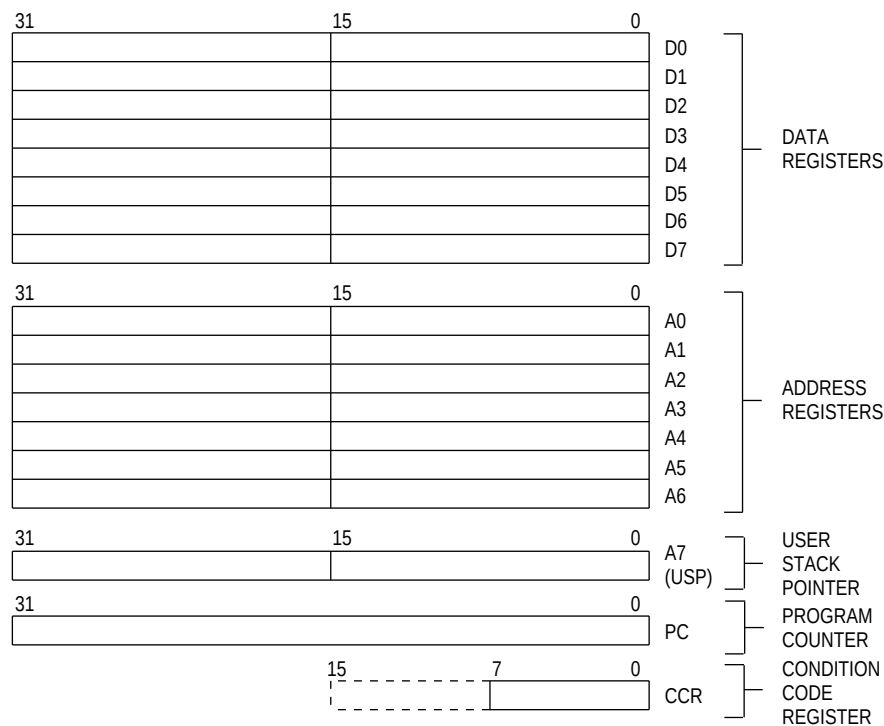


Figure 1-1. M68000 Family User Programming Model

1.1.1 Data Registers (D7 – D0)

These registers are for bit and bit field (1 – 32 bits), byte (8 bits), word (16 bits), long-word (32 bits), and quad-word (64 bits) operations. They also can be used as index registers.

1.1.2 Address Registers (A7 – A0)

These registers can be used as software stack pointers, index registers, or base address registers. The base address registers can be used for word and long-word operations. Register A7 is used as a hardware stack pointer during stacking for subroutine calls and exception handling. In the user programming model, A7 refers to the user stack pointer (USP).

1.1.3 Program Counter

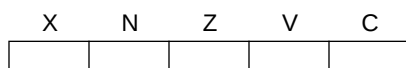
The PC contains the address of the instruction currently executing. During instruction execution and exception processing, the processor automatically increments the contents or places a new value in the PC. For some addressing modes, the PC can be used as a pointer for PC relative addressing.

1.1.4 Condition Code Register

Consisting of five bits, the CCR, the status register's lower byte, is the only portion of the status register (SR) available in the user mode. Many integer instructions affect the CCR, indicating the instruction's result. Program and system control instructions also use certain combinations of these bits to control program and system flow. The condition codes meet two criteria: consistency across instructions, uses, and instances and meaningful results with no change unless it provides useful information.

Consistency across instructions means that all instructions that are special cases of more general instructions affect the condition codes in the same way. Consistency across uses means that conditional instructions test the condition codes similarly and provide the same results whether a compare, test, or move instruction sets the condition codes. Consistency across instances means that all instances of an instruction affect the condition codes in the same way.

The first four bits represent a condition of the result generated by an operation. The fifth bit or the extend bit (X-bit) is an operand for multiprecision computations. The carry bit (C-bit) and the X-bit are separate in the M68000 family to simplify programming techniques that use them (refer to Table 3-18 as an example). In the instruction set definitions, the CCR is illustrated as follows:



X—Extend

Set to the value of the C-bit for arithmetic operations; otherwise not affected or set to a specified result.

N—Negative

Set if the most significant bit of the result is set; otherwise clear.

Z—Zero

Set if the result equals zero; otherwise clear.

V—Overflow

Set if an arithmetic overflow occurs implying that the result cannot be represented in the operand size; otherwise clear.

C—Carry
Set if a carry out of the most significant bit of the operand occurs for an addition, or if a borrow occurs in a subtraction; otherwise clear.

1.2 FLOATING-POINT UNIT USER PROGRAMMING MODEL

The following paragraphs describe the registers for the floating- point unit user programming model. Figure 1-2 illustrates the M68000 family user programming model's floating-point portion for the MC68040 and the MC68881/MC68882 floating-point coprocessors. It contains the following registers:

- 8 Floating-Point Data Registers (FP7 – FP0)
- 16-Bit Floating-Point Control Register (FPCR)
- 32-Bit Floating-Point Status Register (FPSR)
- 32-Bit Floating-Point Instruction Address Register (FPIAR)

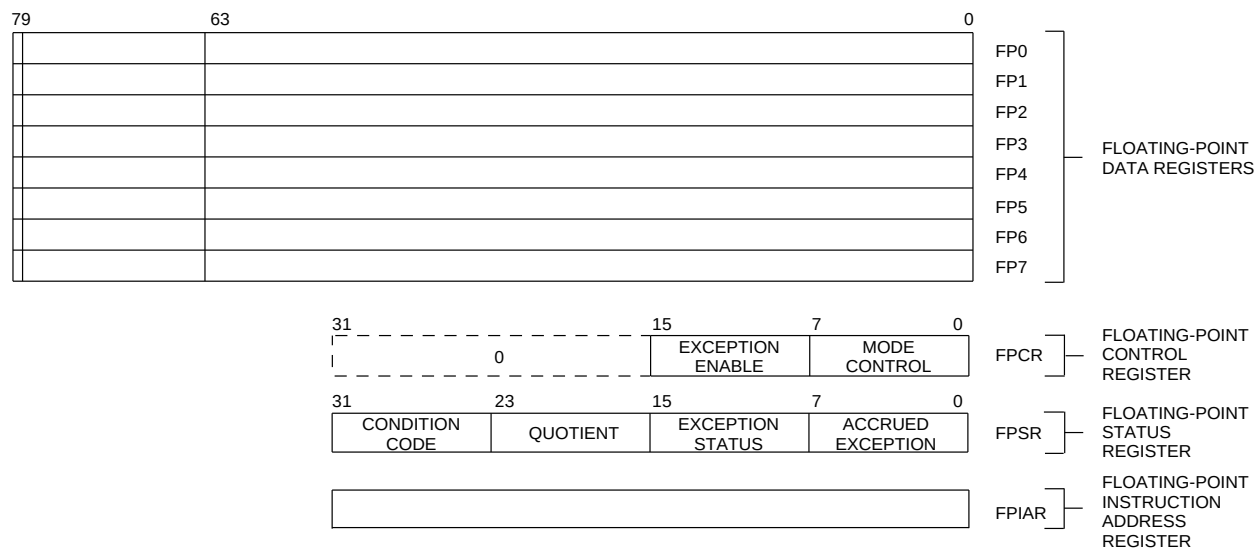


Figure 1-2. M68000 Family Floating-Point Unit User Programming Model

1.2.1 Floating-Point Data Registers (FP7 – FP0)

These floating-point data registers are analogous to the integer data registers for the M68000 family. They always contain extended- precision numbers. All external operands, despite the data format, are converted to extended-precision values before being used in any calculation or being stored in a floating-point data register. A reset or a null-restore operation sets FP7 – FP0 positive, nonsignaling not-a-numbers (NaNs).

1.2.2 Floating-Point Control Register (FPCR)

The FPCR (see Figure 1-3) contains an exception enable (ENABLE) byte and a mode control (MODE) byte. The user can read or write to the FPCR. Motorola reserves bits 31 – 16 for future definition; these bits are always read as zero and are ignored during write operations. The reset function or a restore operation of the null state clears the FPCR. When cleared, this register provides the IEEE 754 Standard for Binary Floating-Point Arithmetic defaults.

1.2.2.1 EXCEPTION ENABLE BYTE. Each bit of the ENABLE byte (see Figure 1-3) corresponds to a floating-point exception class. The user can separately enable traps for each class of floating-point exceptions.

1.2.2.2 MODE CONTROL BYTE. MODE (see Figure 1-3) controls the user-selectable rounding modes and precisions. Zeros in this byte select the IEEE 754 standard defaults. The rounding mode (RND) field specifies how inexact results are rounded, and the rounding precision (PREC) field selects the boundary for rounding the mantissa. Refer to Table 3-21 for encoding information. .

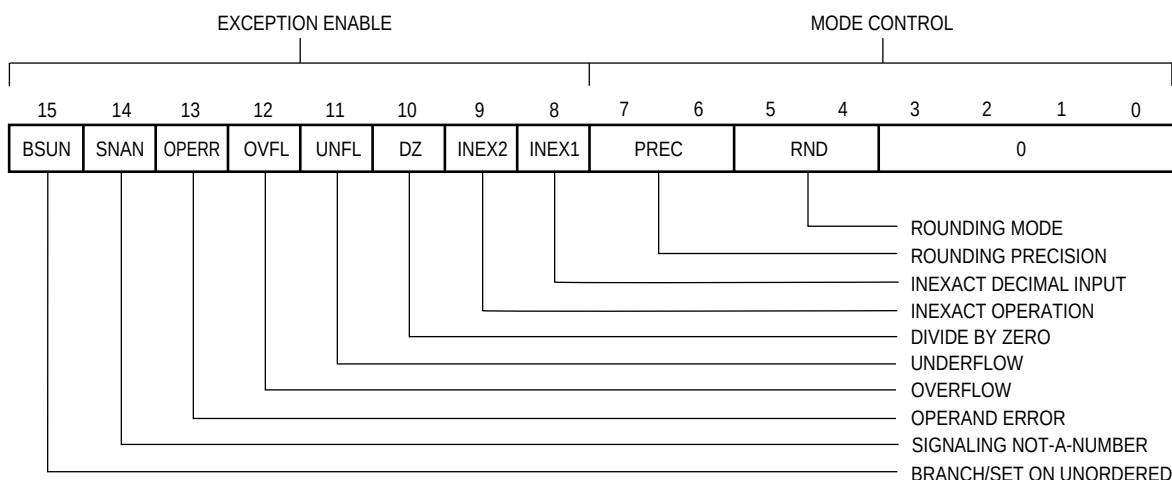


Figure 1-3. Floating-Point Control Register

1.2.3 Floating-Point Status Register (FPSR)

The FPSR (see Figure 1-2) contains a floating-point condition code (FPCC) byte, a floating-point exception status (EXC) byte, a quotient byte, and a floating-point accrued exception (AEXC) byte. The user can read or write to all the bits in the FPSR. Execution of most floating-point instructions modifies this register. The reset function or a restore operation of the null state clears the FPSR.

1.2.3.1 FLOATING-POINT CONDITION CODE BYTE. The FPCC byte, illustrated in Figure 1-4, contains four condition code bits that set after completion of all arithmetic instructions involving the floating-point data registers. The move floating-point data register

to effective address, move multiple floating-point data register, and move system control register instructions do not affect the FPCC. .

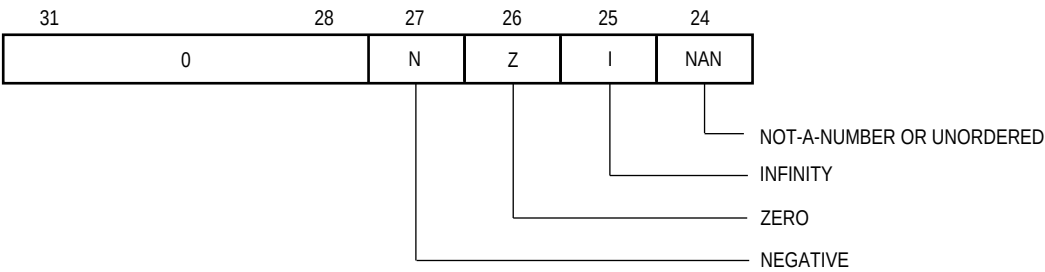


Figure 1-4. FPSR Condition Code Byte

1.2.3.2 QUOTIENT BYTE. The quotient byte contains the seven least significant bits of the unsigned quotient as well as the sign of the entire quotient (see Figure 1-5). The quotient bits can be used in argument reduction for transcendentials and other functions. For example, seven bits are more than enough to figure out the quadrant of a circle in which an operand resides. The quotient bits remain set until the user clears them. .

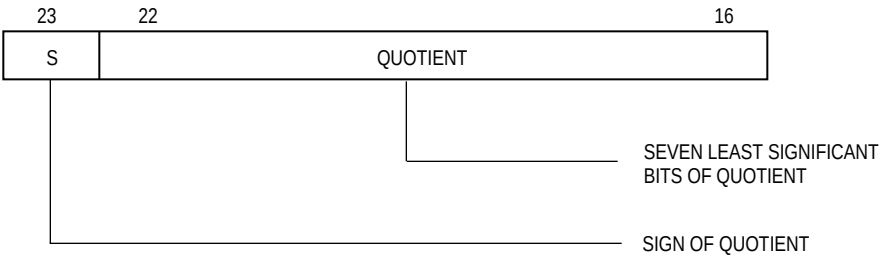


Figure 1-5. FPSR Quotient Code Byte

1.2.3.3 EXCEPTION STATUS BYTE. The EXC byte, illustrated in Figure 1- 6, contains a bit for each floating-point exception that might have occurred during the most recent arithmetic instruction or move operation. This byte is cleared at the start of all operations that generate floating-point exceptions. Operations that do not generate floating-point exceptions do not clear this byte. An exception handler can use this byte to determine which floating-point exception(s) caused a trap. .

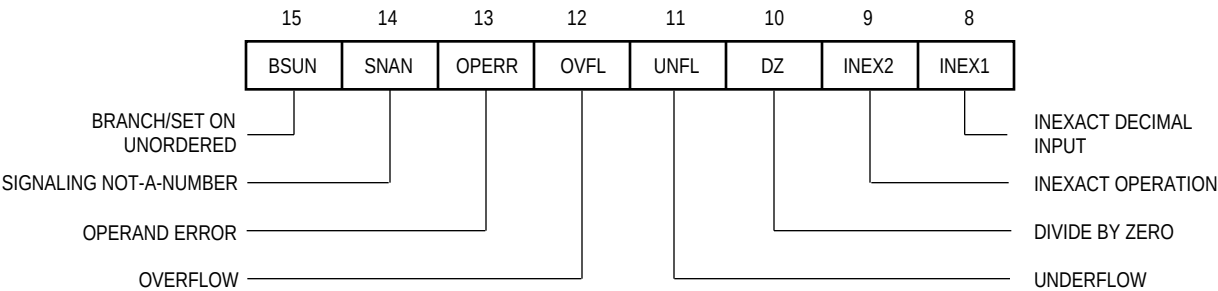


Figure 1-6. FPSR Exception Status Byte

1.2.3.4 ACCRUED EXCEPTION BYTE. The AEXC byte contains five exception bits (see Figure 1-7) required by the IEEE 754 standard for trap disabled operations. These exceptions are logical combinations of the bits in the EXC byte. The AEXC byte contains a history of all floating-point exceptions that have occurred since the user last cleared the AEXC byte. In normal operations, only the user clears this byte by writing to the FPSR; however, a reset or a restore operation of the null state can also clear the AEXC byte.

Many users elect to disable traps for all or part of the floating-point exception classes. The AEXC byte makes it unnecessary to poll the EXC byte after each floating-point instruction. At the end of most operations (FMOVEM and FMOVE excluded), the bits in the EXC byte are logically combined to form an AEXC value that is logically ORed into the existing AEXC byte. This operation creates "sticky" floating-point exception bits in the AEXC byte that the user needs to poll only once—i.e., at the end of a series of floating-point operations.

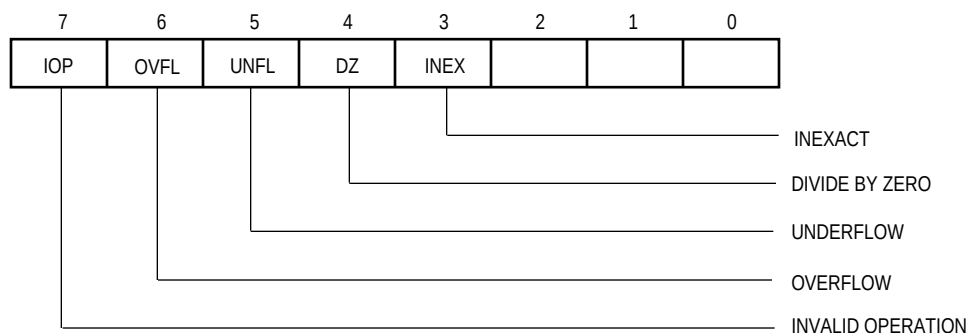


Figure 1-7. FPSR Accrued Exception Byte

Setting or clearing the AEXC bits neither causes nor prevents an exception. The following equations show the comparative relationship between the EXC byte and AEXC byte. Comparing the current value in the AEXC bit with a combination of bits in the EXC byte derives a new value in the corresponding AEXC bit. These equations apply to setting the AEXC bits at the end of each operation affecting the AEXC byte:

New AEXC Bit	= Old AEXC Bit	V	EXC Bits
IOP	= IOP	V	(SNAN V OPERR)
OVFL	= OVFL	V	(OVFL)
UNFL	= UNFL	V	(UNFL L INEX2)
DZ	= DZ	V	(DZ)
INEX	= INEX	V	(INEX1 V INEX2 V OVFL)

1.2.4 Floating-Point Instruction Address Register (FPIAR)

The integer unit can be executing instructions while the FPU is simultaneously executing a floating-point instruction. Additionally, the FPU can concurrently execute two floating-point instructions. Because of this nonsequential instruction execution, the PC value stacked by the FPU, in response to a floating-point exception trap, may not point to the offending instruction.

For the subset of the FPU instructions that generate exception traps, the 32-bit FPIAR is loaded with the logical address of the instruction before the processor executes it. The floating-point exception handler can use this address to locate the floating-point instruction that caused an exception. Since the FPU FMOVE to/from the FPCR, FPSR, or FPIAR and FMOVEM instructions cannot generate floating-point exceptions, these instructions do not modify the FPIAR. A reset or a null-restore operation clears the FPIAR.

1.3 SUPERVISOR PROGRAMMING MODEL

System programmers use the supervisor programming model to implement sensitive operating system functions—e.g., I/O control and memory management unit (MMU) subsystems. The following paragraphs briefly describe the registers in the supervisor programming model. They can only be accessed via privileged instructions. Table 1-1 lists the supervisor registers and the processors not related to paged memory management. For information concerning page memory management programming, refer to the device-specific user's manual. Table 1-2 lists the supervisor registers and the processors related to paged memory management.

**Table 1-1. Supervisor Registers
Not Related To Paged Memory Management**

Registers	Devices								
	68000 68008 68HC000 68HC001 68EC000	68010	68020 68EC020	CPU32	68030	68EC030	68040	68EC040	68LC040
AC1, AC0						X			
ACUSR						X			
CAAR			X		X	X			
CACR			X		X	X	X	X	X
DACR1, DACR0								X	
DFC		X	X	X	X	X	X	X	X
DTT1, DTT0							X		X
IACR1, IACR0								X	
ITT1, ITT0							X		X
MSP			X		X	X	X	X	X
SFC		X	X	X	X	X	X	X	X
SR	X	X	X	X	X	X	X	X	X
SSP/ISP	X	X	X	X	X	X	X	X	X
TT1, TT0					X				
VBR		X	X	X	X	X	X	X	X

AC1, AC0 = Access Control Registers	ITT1, ITT0 = Instruction Transparent
ACUSR = Access Control Unit Status Register	Translation Registers
CAAR = Cache Address Register	MSP = Master Stack Pointer Register
CACR = Cache Control Register	SFC = Source Function Code Register
DACR1, DACR0 = Data Access Control Registers	SR = Status Register
DFC = Destination Function Code Register	SSP/ISP = Supervisor and Interrupt Stack Pointer
DTT1, DTT0 = Data Transparent Translation Registers	TT1, TT0 = Transparent Translation Registers
IACR1, IACR0 = Instruction Access Control Registers	VBR = Vector Base Register

**Table 1-2. Supervisor Registers
Related To Paged Memory Management**

Registers	Devices			
	68851	68030	68040	68LC040
AC	x			
CAL	x			
CRP	x	x		
DRP	x			
PCSR	x			
PMMUSR, MMUSR	x	x	x	x
SCC	x			
SRP	x	x	x	x
TC	x	x	x	x
URP			x	x
VAL	x			

- AC = Access Control Register
- CAL = Current Access Level Register
- CRP = CPU Root Pointer
- DRP = DMA Root Pointer
- PCSR = PMMU Control Register
- PMMUSR = Paged Memory Management Unit Status Register
- MMUSR = Memory Management Unit Status Register
- SCC = Stack Change Control Register
- SRP = Supervisor Root Pointer Register
- TC = Translation Control Register
- URP = User Root Pointer
- VAL = Valid Access Level Register

1.3.1 Address Register 7 (A7)

In the supervisor programming model register, A7 refers to the interrupt stack pointer, A7'(ISP) and the master stack pointer, A7" (MSP). The supervisor stack pointer is the active stack pointer (ISP or MSP). For processors that do not support ISP or MSP, the system stack is the system stack pointer (SSP). The ISP and MSP are general- purpose address registers for the supervisor mode. They can be used as software stack pointers, index registers, or base address registers. The ISP and MSP can be used for word and long-word operations.

1.3.2 Status Register

Figure 1-8 illustrates the SR, which stores the processor status and contains the condition codes that reflect the results of a previous operation. In the supervisor mode, software can access the full SR, including the interrupt priority mask and additional control bits. These bits indicate the following states for the processor: one of two trace modes (T1, T0), supervisor or user mode (S), and master or interrupt mode (M). For the MC68000, MC68EC000, MC68008, MC68010, MC68HC000, MC68HC001, and CPU32, only one trace mode

supported, where T0 is always zero, and only one system stack where the M-bit is always zero. I2, I1, and I0 define the interrupt mask level.

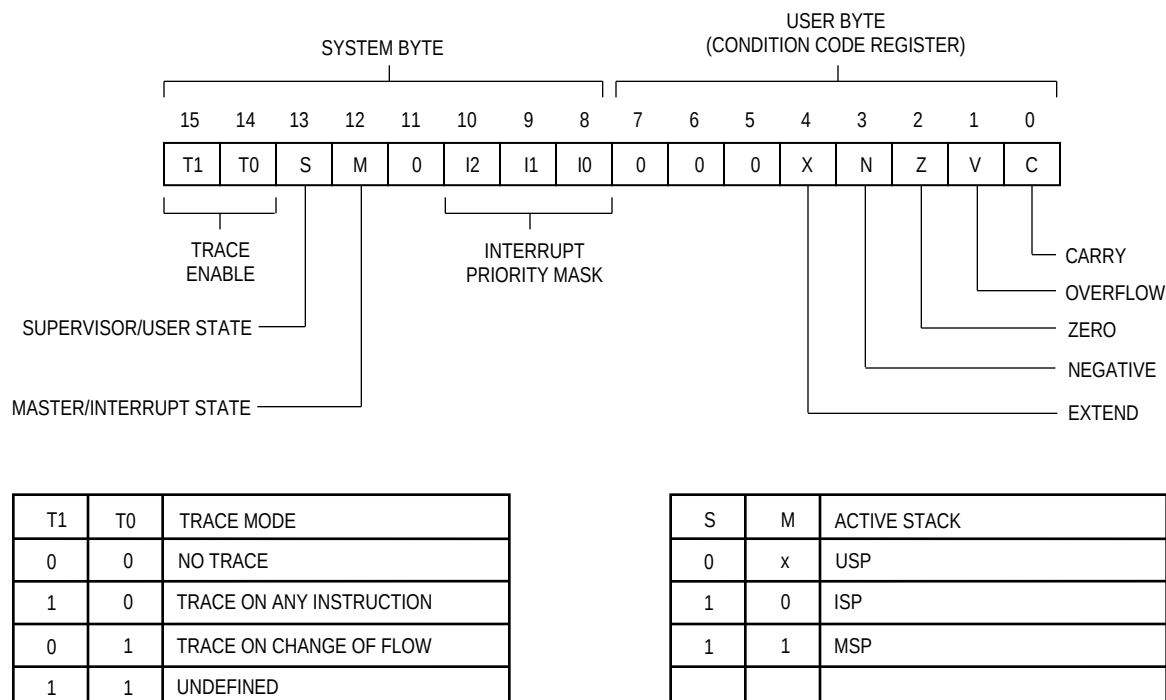


Figure 1-8. Status Register

1.3.3 Vector Base Register (VBR)

The VBR contains the base address of the exception vector table in memory. The displacement of an exception vector adds to the value in this register, which accesses the vector table.

1.3.4 Alternate Function Code Registers (SFC and DFC)

The alternate function code registers contain 3-bit function codes. Function codes can be considered extensions of the 32-bit logical address that optionally provides as many as eight 4-Gbyte address spaces. The processor automatically generates function codes to select address spaces for data and programs at the user and supervisor modes. Certain instructions use SFC and DFC to specify the function codes for operations.

1.3.5 Acu Status Register (MC68EC030 only)

The access control unit status register (ACUSR) is a 16-bit register containing the status information returned by execution of the PTEST instruction. The PTEST instruction searches the access control (AC) registers to determine a match for a specified address. A match in either or both of the AC registers sets bit 6 in the ACUSR. All other bits in the ACUSR are undefined and must not be used.

1.3.6 Transparent Translation/access Control Registers

Transparent translation is actually a misnomer since the whole address space transparently translates in an embedded control environment with no on-chip MMU present as well as in processors that have built-in MMUs. For processors that have built-in MMUs, such as the MC68030, MC68040, and MC68LC040, the transparent translation (TT) registers define blocks of logical addresses that are transparently translated to corresponding physical addresses. These registers are independent of the on-chip MMU. For embedded controllers, such as the MC68EC030 and MC68EC040, the access control registers (AC) are similar in function to the TT registers but just named differently. The AC registers, main function are to define blocks of address space that control address space properties such as cachability. The following paragraphs describe these registers.

NOTE

For the paged MMU related supervisor registers, please refer to the appropriate user’s manual for specific programming detail.

1.3.6.1 TRANSPARENT TRANSLATION/ACCESS CONTROL REGISTER FIELDS FOR THE M68030. Figure 1-9 illustrates the MC68030 transparent translation/MC68EC030 access control register format.

31								24	23	16					
ADDRESS BASE								ADDRESS MASK							
E	0	0	0	0	CI	R/W	RWM	0	FC BASE		0	FC MASK			
15	14	13	12	11	10	9	8	7	6	4	3	2	0		

Figure 1-9. MC68030 Transparent Translation/MC68EC030 Access Control Register Format

Address Base

This 8-bit field is compared with address bits A31 – A24. Addresses that match in this comparison (and are otherwise eligible) are transparently translated/access controlled.

Address Mask

This 8-bit field contains a mask for the address base field. Setting a bit in this field causes the corresponding bit of the address base field to be ignored. Blocks of memory larger than 16 Mbytes can be transparently translated/accessed controlled by setting some logical address mask bits to ones. The low-order bits of this field normally are set to define contiguous blocks larger than 16 Mbytes, although this is not required.

E—Enable

- 0 = Transparent translation/access control disabled
- 1 = Transparent translation/access control enabled

CI—Cache Inhibit

- 0 = Caching allowed
- 1 = Caching inhibited

R/W—Read/Write

- 0 = Only write accesses permitted
- 1 = Only read accesses permitted

R/WM—Read/Write Mask

- 0 = R/W field used
- 1 = R/W field ignored

FC BASE—Function Code Base

This 3-bit field defines the base function code for accesses to be transparently translated with this register. Addresses with function codes that match the FC BASE field (and are otherwise eligible) are transparently translated.

FC MASK—Function Code Mask

This 3-bit field contains a mask for the FC BASE field. Setting a bit in this field causes the corresponding bit of the FC BASE field to be ignored.

1.3.6.2 TRANSPARENT TRANSLATION/ACCESS CONTROL REGISTER FIELDS FOR THE M68040. Figure 1-10 illustrates the MC68040 and MC68LC040 transparent translation/ MC68EC040 access control register format.

31								24		23		16							
ADDRESS BASE								ADDRESS MASK											
E	S FIELD			0	0	0	U1	U0	0	CM		0	0	W	0	0			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				

Figure 1-10. MC68040 and MC68LC040 Transparent Translation/MC68EC040 Access Control Register Format

Address Base

This 8-bit field is compared with address bits A31 – A24. Addresses that match in this comparison (and are otherwise eligible) are transparently translated/access controlled.

Address Mask

This 8-bit field contains a mask for the address base field. Setting a bit in this field causes the corresponding bit in the address base field to be ignored. Blocks of memory larger than 16 Mbytes can be transparently translated/access controlled by setting some logical address mask bits to ones. The low-order bits of this field normally are set to define contiguous blocks larger than 16 Mbytes, although this not required.

E—Enable

This bit enables and disables transparent translation/access control of the block defined by this register.

- 0 = Transparent translation/access control disabled
- 1 = Transparent translation/access control enabled

S—Supervisor/User Mode

This field specifies the use of the FC2 in matching an address.

- 00 = Match only if FC2 is 0 (user mode access)
- 01 = Match only if FC2 is 1 (supervisor mode access)
- 1X = Ignore FC2 when matching

U1, U2—User Page Attributes

The MC68040, MC68E040, MC68LC040 do not interpret these user-defined bits. If an external bus transfer results from the access, U0 and U1 are echoed to the UPA0 and UPA1 signals, respectively.

CM—Cache Mode

This field selects the cache mode and access serialization for a page as follows:

- 00 = Cachable, Writethrough
- 01 = Cachable, Copyback
- 10 = Noncachable, Serialized
- 11 = Noncachable

W—Write Protect

This bit indicates if the block is write protected. If set, write and read-modify-write accesses are aborted as if the resident bit in a table descriptor were clear.

- 0 = Read and write accesses permitted
- 1 = Write accesses not permitted

1.4 INTEGER DATA FORMATS

The operand data formats supported by the integer unit, as listed in Table 1-3, include those supported by the MC68030 plus a new data format (16-byte block) for the MOVE16 instruction. Integer unit operands can reside in registers, memory, or instructions themselves. The operand size for each instruction is either explicitly encoded in the instruction or implicitly defined by the instruction operation.

Table 1-3. Integer Data Formats

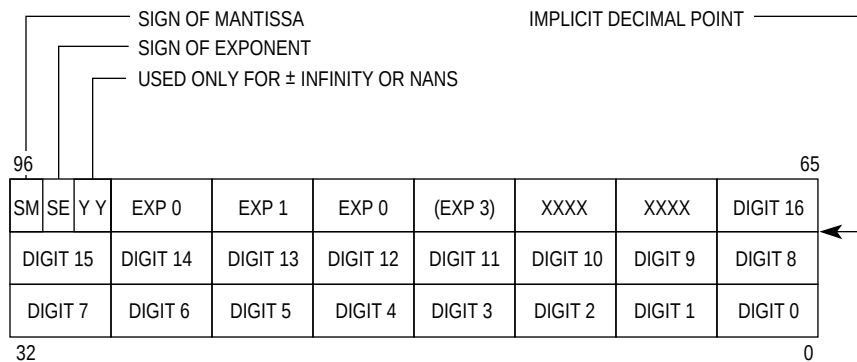
Operand Data Format	Size	Notes
Bit	1 Bit	—
Bit Field	1 – 32 Bits	Field of Consecutive Bit
Binary-Coded Decimal (BCD)	8 Bits	Packed: 2 Digits/Byte; Unpacked: 1 Digit/Byte
Byte Integer	8 Bits	—
Word Integer	16 Bits	—
Long-Word Integer	32 Bits	—
Quad-Word Integer	64 Bits	Any Two Data Registers
16-Byte	128 Bits	Memory Only, Aligned to 16- Byte Boundary

1.5 FLOATING-POINT DATA FORMATS

The following paragraphs describe the FPU's operand data formats. The FPU supports seven data formats. There are three signed binary integer formats (byte, word, and long word) that are identical to those supported by the integer unit. The FPU supports the use of the packed decimal real format. The MC68881 and MC68882 support this format in hardware and the processors starting with the MC68040 support it in software. The FPU also supports three binary floating- point formats (single, double, and extended precision) that fully comply with the IEEE 754 standard. All references in this manual to extended-precision format imply the double-extended-precision format defined by the IEEE 754 standard.

1.5.1 Packed Decimal Real Format

Figure 1-11 illustrates the packed decimal real format which is three long words consisting of a 3-digit base 10 exponent and a 17-digit base 10 mantissa. The first two long words, digits 15 – 0, are 64 bits and map directly to bit positions 63 – 0 of the extended-precision real format. There are two separate sign bits, one for the exponent, the other for the mantissa. An extra exponent (EXP3) is defined for overflows that can occur when converting from the extended-precision real format to the packed decimal real format.



NOTE: XXXX indicates "don't care", which is zero when written and ignored when read.

Figure 1-11. Packed Decimal Real Format

1.5.2 Binary Floating-Point Formats

Figure 1-12 illustrates the three binary floating-point data formats. The exponent in the three binary floating-point formats is an unsigned binary integer with an implied bias added to it. When subtracting the bias from the exponent's value, the result represents a signed two's complement power of two. This yields the magnitude of a normalized floating-point number when multiplied by the mantissa. A program can execute a CMP instruction that compares floating-point numbers in memory using biased exponents, despite the absolute magnitude of the exponents.

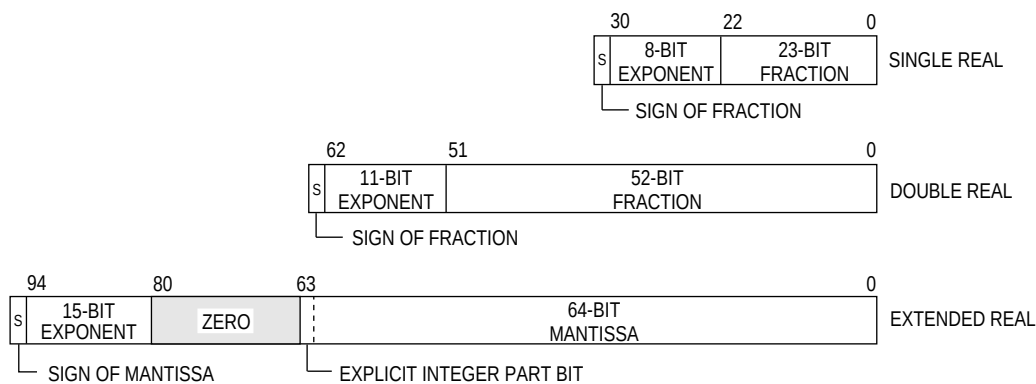


Figure 1-12. Binary Floating-Point Data Formats

Data formats for single- and double-precision numbers differ slightly from those for extended-precision numbers in the representation of the mantissa. For all three precisions, a normalized mantissa is always in the range (1.0...2.0). The extended-precision data format represents the entire mantissa, including the explicit integer part bit. Single- and double-precision data formats represent only a fractional portion of the mantissa (the fraction) and always imply the integer part as one.

The IEEE 754 standard has created the term significand to bridge the difference between mantissa and fraction and to avoid the historical implications of the term mantissa. The IEEE 754 standard defines a significand as the component of a binary floating-point number that includes an explicit or implicit leading bit to the left of the implied binary point. However, this manual uses the term mantissa for extended-precision formats and fraction for single- and double-precision formats instead of the IEEE term significand.

NOTE

This section specifies ranges using traditional set notation with the format "bound...bound" specifying the boundaries of the range. The bracket types enclosing the range define whether the endpoint is inclusive or exclusive. A square bracket indicates inclusive, and a parenthesis indicates exclusive. For example, the range specification "[1.0...2.0]" defines the range of numbers greater than or equal to 1.0 and less than or equal to 2.0. The range specification "(0.0... + inf)" defines the range of numbers greater than 0.0 and less than positive infinity, but not equal to.

1.6 FLOATING-POINT DATA TYPES

Each floating-point data format supports five, unique, floating-point data types: 1) normalized numbers, 2) denormalized numbers, 3) zeros, 4) infinities, and 5) NaNs. Exponent values in each format represent these special data types. The normalized data type never uses the maximum or minimum exponent value for a given format, except the extended-precision format. The packed decimal real data format does not support denormalized numbers.

There is a subtle difference between the definition of an extended-precision number with an exponent equal to zero and a single- or double-precision number with an exponent equal to zero. The zero exponent of a single- or double-precision number denormalizes the number's definition, and the implied integer bit is zero. An extended-precision number with an exponent of zero may have an explicit integer bit equal to one. This results in a normalized number, though the exponent is equal to the minimum value. For simplicity, the following discussion treats all three floating-point formats in the same manner, where an exponent value of zero identifies a denormalized number. However, remember the extended-precision format can deviate from this rule.

1.6.1 Normalized Numbers

Normalized numbers encompass all numbers with exponents laying between the maximum and minimum values. Normalized numbers can be positive or negative. For normalized numbers in single and double precision the implied integer bit is one. In extended precision, the mantissa's MSB, the explicit integer bit, can only be a one (see Figure 1-13); and the exponent can be zero.

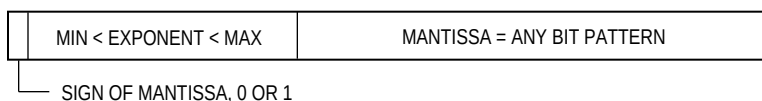


Figure 1-13. Normalized Number Format

1.6.2 Denormalized Numbers

Denormalized numbers represent real values near the underflow threshold. The detection of the underflow for a given data format and operation occurs when the result's exponent is less than or equal to the minimum exponent value. Denormalized numbers can be positive or negative. For denormalized numbers in single and double precision the implied integer bit is a zero. In extended precision, the mantissa's MSB, the explicit integer bit, can only be a zero (see Figure 1-14).



Figure 1-14. Denormalized Number Format

Traditionally, the detection of underflow causes floating-point number systems to perform a "flush-to-zero". This leaves a large gap in the number line between the smallest magnitude normalized number and zero. The IEEE 754 standard implements gradual underflows: the result mantissa is shifted right (denormalized) while the result exponent is incremented until reaching the minimum value. If all the mantissa bits of the result are shifted off to the right during this denormalization, the result becomes zero. Usually a gradual underflow limits the potential underflow damage to no more than a round-off error. This underflow and denormalization description ignores the effects of rounding and the user-selectable rounding modes. Thus, the large gap in the number line created by "flush-to-zero" number systems is filled with representable (denormalized) numbers in the IEEE "gradual underflow" floating-point number system.

Since the extended-precision data format has an explicit integer bit, a number can be formatted with a nonzero exponent, less than the maximum value, and a zero integer bit. The IEEE 754 standard does not define a zero integer bit. Such a number is an unnormalized number. Hardware does not directly support denormalized and unnormalized numbers, but implicitly supports them by trapping them as unimplemented data types, allowing efficient conversion in software.

1.6.3 Zeros

Zeros can be positive or negative and represent the real values + 0.0 and – 0.0 (see Figure 1-15).

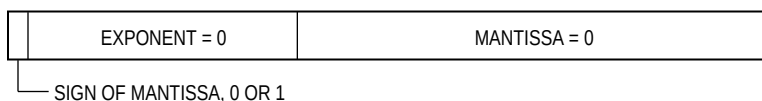


Figure 1-15. Zero Format

1.6.4 Infinities

Infinities can be positive or negative and represent real values that exceed the overflow threshold. A result's exponent greater than or equal to the maximum exponent value indicates the overflow for a given data format and operation. This overflow description ignores the effects of rounding and the user-selectable rounding models. For single- and double-precision infinities the fraction is a zero. For extended-precision infinities, the mantissa's MSB, the explicit integer bit, can be either one or zero (see Figure 1-16).

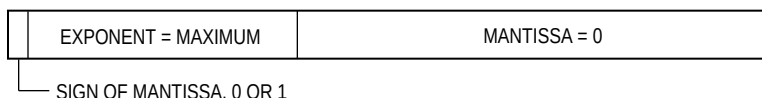


Figure 1-16. Infinity Format

1.6.5 Not-A-Numbers

When created by the FPU, NaNs represent the results of operations having no mathematical interpretation, such as infinity divided by infinity. All operations involving a NaN operand as an input return a NaN result. When created by the user, NaNs can protect against uninitialized variables and arrays or represent user-defined data types. For extended-precision NaNs, the mantissa's MSB, the explicit integer bit, can be either one or zero (see Figure 1-17).

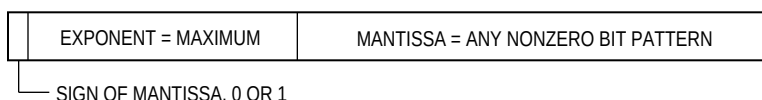


Figure 1-17. Not-A-Number Format

The FPU implements two different types of NaNs identified by the value of the MSB of the mantissa for single- and double-precision, and the MSB of the mantissa minus one for extended-precision. If the bit is set, it is a nonsignaling NaN, otherwise, it is an SNAN. An

SNAN can be used as an escape mechanism for a user-defined, non-IEEE data type. The FPU never creates an SNAN resulting from an operation.

The IEEE specification defines NAN processing used as an input to an operation. A nonsignaling NAN must be returned when using an SNAN as an input and there is a disabled SNAN trap. The FPU does this by using the source SNAN, setting the MSB of the mantissa, and storing the resulting nonsignaling NAN in the destination. Because of the IEEE formats for NANs, the result of setting an SNAN MSB is always a nonsignaling NAN.

When the FPU creates a NAN, the NAN always contains the same bit pattern in the mantissa. All bits of the mantissa are ones for any precision. When the user creates a NAN, any nonzero bit pattern can be stored in the mantissa.

1.6.6 Data Format and Type Summary

Tables 1-4 through 1-6 summarize the data type specifications for single-, double-, and extended-precision data formats. Packed decimal real formats support all data types except denormalized numbers. Table 1-7 summarizes the data types for the packed decimal real format.

Table 1-4. Single-Precision Real Format Summary Data Format

Data Format	
31 30 23 22 0 s e f	
Field Size In Bits	
Sign (s)	1
Biased Exponent (e)	8
Fraction (f)	23
Total	32
Interpretation of Sign	
Positive Fraction	$s = 0$
Negative Fraction	$s = 1$
Normalized Numbers	
Bias of Biased Exponent	+127 (\$7F)
Range of Biased Exponent	$0 < e < 255$ (\$FF)
Range of Fraction	Zero or Nonzero
Fraction	1.f
Relation to Representation of Real Numbers	$(-1)^s \times 2^{e-127} \times 1.f$
Denormalized Numbers	
Biased Exponent Format Minimum	0 (\$00)
Bias of Biased Exponent	+126 (\$7E)
Range of Fraction	Nonzero
Fraction	0.f
Relation to Representation of Real Numbers	$(-1)^s \times 2^{-126} \times 0.f$
Signed Zeros	
Biased Exponent Format Minimum	0 (\$00)
Fraction	$0.f = 0.0$
Signed Infinities	
Biased Exponent Format Maximum	255 (\$FF)
Fraction	$0.f = 0.0$
NaNs	
Sign	Don't Care
Biased Exponent Format Maximum	255 (\$FF)
Fraction	Nonzero
Representation of Fraction	Nonsignaling Signaling Nonzero Bit Pattern Created by User Fraction When Created by FPCP 0.1xxxx...xxxx 0.0xxxx...xxxx xxxxx...xxxx 11111...1111
Approximate Ranges	
Maximum Positive Normalized	3.4×10^{38}
Minimum Positive Normalized	1.2×10^{-38}
Minimum Positive Denormalized	1.4×10^{-45}

Table 1-5. Double-Precision Real Format Summary

Data Format	
63 62 52 51 0 s e f	
Field Size (in Bits)	
Sign (s)	1
Biased Exponent (e)	11
Fraction (f)	52
Total	64
Interpretation of Sign	
Positive Fraction	s = 0
Negative Fraction	s = 1
Normalized Numbers	
Bias of Biased Exponent	+1023 (\$3FF)
Range of Biased Exponent	0 < e < 2047 (\$7FF)
Range of Fraction	Zero or Nonzero
Fraction	1.f
Relation to Representation of Real Numbers	$(-1)^s \times 2^{e-1023} \times 1.f$
Denormalized Numbers	
Biased Exponent Format Minimum	0 (\$000)
Bias of Biased Exponent	+1022 (\$3FE)
Range of Fraction	Nonzero
Fraction	0.f
Relation to Representation of Real Numbers	$(-1)^s \times 2^{-1022} \times 0.f$
Signed Zeros	
Biased Exponent Format Minimum	0 (\$00)
Fraction (Mantissa/Significand)	0.f = 0.0
Signed Infinities	
Biased Exponent Format Maximum	2047 (\$7FF)
Fraction	0.f = 0.0
NaNs	
Sign	0 or 1
Biased Exponent Format Maximum	255 (\$7FF)
Fraction	Nonzero
Representation of Fraction	
Nonsignaling	1xxxx...xxxx
Signaling	0xxxx...xxxx
Nonzero Bit Pattern Created by User	xxxxx...xxxx
Fraction When Created by FPCP	11111...1111
Approximate Ranges	
Maximum Positive Normalized	18×10^{308}
Minimum Positive Normalized	2.2×10^{-308}
Minimum Positive Denormalized	4.9×10^{-324}

Table 1-6. Extended-Precision Real Format Summary

Data Format	
95 94 80 79 64 63 62 0 s e z i f	
Field Size (in Bits)	
Sign (s)	1
Biased Exponent (e)	15
Zero, Reserved (u)	16
Explicit Integer Bit (j)	1
Mantissa (f)	63
Total	96
Interpretation of Unused Bits	
Input	Don't Care
Output	All Zeros
Interpretation of Sign	
Positive Mantissa	s = 0
Negative Mantissa	s = 1
Normalized Numbers	
Bias of Biased Exponent	+16383 (\$3FFF)
Range of Biased Exponent	$0 < e < 32767$ (\$7FFF)
Explicit Integer Bit	1
Range of Mantissa	Zero or Nonzero
Mantissa (Explicit Integer Bit and Fraction)	1.f
Relation to Representation of Real Numbers	$(-1)^s \times 2^{e-16383} \times 1.f$
Denormalized Numbers	
Biased Exponent Format Minimum	0 (\$0000)
Bias of Biased Exponent	+16383 (\$3FFF)
Explicit Integer Bit	0
Range of Mantissa	Nonzero
Mantissa (Explicit Integer Bit and Fraction)	0.f
Relation to Representation of Real Numbers	$(-1)^s \times 2^{-16383} \times 0.f$
Signed Zeros	
Biased Exponent Format Minimum	0 (\$0000)
Mantissa (Explicit Integer Bit and Fraction)	0.0
Signed Infinities	
Biased Exponent Format Maximum	32767 (\$7FFF)
Explicit Integer Bit	Don't Care
Mantissa (Explicit Integer Bit and Fraction)	x.000...0000

Table 1-6. Extended-Precision Real Format Summary (Continued)

NANs	
Sign	Don't Care
Explicit Integer Bit	Don't Care
Biased Exponent Format Maximum	32767 (\$7FFF)
Mantissa	Nonzero
Representation of Fraction	
Nonsignaling	x.1xxxx...xxxx
Signaling	x.0xxxx...xxxx
Nonzero Bit Pattern Created by User	x.xxxxx...xxxx
Fraction When Created by FPCP	1.1111...1111
Approximate Ranges	
Maximum Positive Normalized	1.2×10^{4932}
Minimum Positive Normalized	1.7×10^{-4932}
Minimum Positive Denormalized	3.7×10^{4951}

Table 1-7. Packed Decimal Real Format Summary

Data Type	SM	SE	Y	Y	3-Digit Exponent	1-Digit Integer	16-Digit Fraction
±Infinity	0/1	1	1	1	\$FFF	\$XXXX	\$00...00
±NAN	0/1	1	1	1	\$FFF	\$XXXX	Nonzero
±SNAN	0/1	1	1	1	\$FFF	\$XXXX	Nonzero
+Zero	0	0/1	X	X	\$000–\$999	\$XXX0	\$00...00
–Zero	1	0/1	X	X	\$000–\$999	\$XXX0	\$00...00
+In-Range	0	0/1	X	X	\$000–\$999	\$XXX0–\$XXX9	\$00...01–\$99...99
–In-Range	1	0/1	X	X	\$000–\$999	\$XXX0–\$XXX9	\$00...01–\$99...99

A packed decimal real data format with the SE and both Y bits set, an exponent of \$FFF, and a nonzero 16-bit decimal fraction is a NAN. When the FPU uses this format, the fraction of the NAN is moved bit- by-bit into the extended-precision mantissa of a floating-point data register. The exponent of the register is set to signify a NAN, and no conversion occurs. The MSB of the most significant digit in the decimal fraction (the MSB of digit 15) is a don't care, as in extended-precision NANs, and the MSB of minus one of digit 15 is the SNAN bit. If the NAN bit is a zero, then it is an SNAN.

If a non-decimal digit (\$A – \$F) appears in the exponent of a zero, the number is a true zero. The FPU does not detect non-decimal digits in the exponent, integer, or fraction digits of an in-range packed decimal real data format. These non-decimal digits are converted to binary in the same manner as decimal digits; however, the result is probably useless although it is repeatable. Since an in-range number cannot overflow or underflow when converted to extended precision, conversion from the packed decimal real data format always produces normalized extended-precision numbers.

1.7 ORGANIZATION OF DATA IN REGISTERS

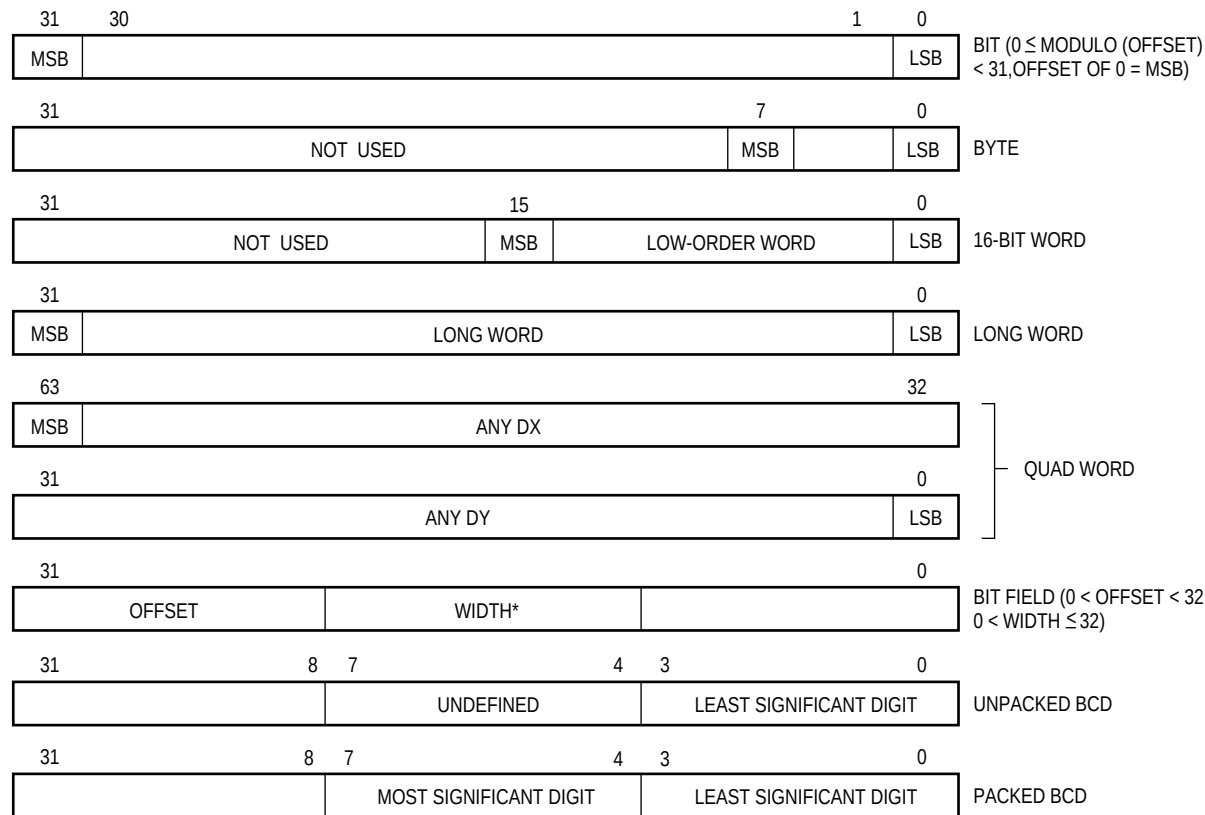
The following paragraphs describe data organization within the data, address, and control registers.

1.7.1 Organization of Integer Data Formats in Registers

Each integer data register is 32 bits wide. Byte and word operands occupy the lower 8- and 16-bit portions of integer data registers, respectively. Long- word operands occupy the entire 32 bits of integer data registers. A data register that is either a source or destination operand only uses or changes the appropriate lower 8 or 16 bits (in byte or word operations, respectively). The remaining high-order portion does not change and goes unused. The address of the least significant bit (LSB) of a long-word integer is zero, and the MSB is 31. For bit fields, the address of the MSB is zero, and the LSB is the width of the register minus one (the offset). If the width of the register plus the offset is greater than 32, the bit field wraps around within the register. Figure 1-18 illustrates the organization of various data formats in the data registers.

An example of a quad word is the product of a 32-bit multiply or the quotient of a 32-bit divide operation (signed and unsigned). Quad words may be organized in any two integer data registers without restrictions on order or pairing. There are no explicit instructions for the management of this data format, although the MOVEM instruction can be used to move a quad word into or out of registers.

Binary-coded decimal (BCD) data represents decimal numbers in binary form. Although there are many BCD codes, the BCD instructions of the M68000 family support two formats, packed and unpacked. In these formats, the LSBs consist of a binary number having the numeric value of the corresponding decimal number. In the unpacked BCD format, a byte defines one decimal number that has four LSBs containing the binary value and four undefined MSBs. Each byte of the packed BCD format contains two decimal numbers; the least significant four bits contain the least significant decimal number and the most significant four bits contain the most significant decimal number.



* IF WIDTH + OFFSET > 32, BIT FIELD WRAPS AROUND WITHIN THE REGISTER.

Figure 1-18. Organization of Integer Data Formats in Data Registers

Because address registers and stack pointers are 32 bits wide, address registers cannot be used for byte-size operands. When an address register is a source operand, either the low-order word or the entire long-word operand is used, depending upon the operation size. When an address register is the destination operand, the entire register becomes affected, despite the operation size. If the source operand is a word size, it is sign-extended to 32 bits and then used in the operation to an address register destination. Address registers are primarily for addresses and address computation support. The instruction set includes instructions that add to, compare, and move the contents of address registers. Figure 1-19 illustrates the organization of addresses in address registers.

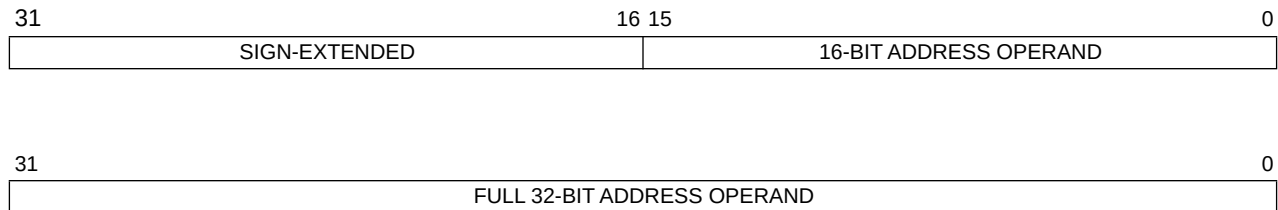


Figure 1-19. Organization of Integer Data Formats in Address Registers

Control registers vary in size according to function. Some control registers have undefined bits reserved for future definition by Motorola. Those particular bits read as zeros and must be written as zeros for future compatibility.

All operations to the SR and CCR are word-size operations. For all CCR operations, the upper byte is read as all zeros and is ignored when written, despite privilege mode. The alternate function code registers, supervisor function code (SFC) and data function code (DFC), are 32-bit registers with only bits 0P2 implemented. These bits contain the address space values for the read or write operands of MOVES, PFLUSH, and PTEST instructions. Values transfer to and from the SFC and DFC by using the MOVEC instruction. These are long-word transfers; the upper 29 bits are read as zeros and are ignored when written.

1.7.2 Organization of Integer Data Formats in Memory

The byte-addressable organization of memory allows lower addresses to correspond to higher order bytes. The address N of a long-word data item corresponds to the address of the highest order word/Us MSB. The lower order word is located at address N + 2, leaving the LSB at address N + 3 (see Figure 1-20). Organization of data formats in memory is consistent with the M68000 family data organization. The lowest address (nearest \$00000000) is the location of the MSB, with each successive LSB located at the next address (N + 1, N + 2, etc.). The highest address (nearest \$FFFFFFF) is the location of the LSB.

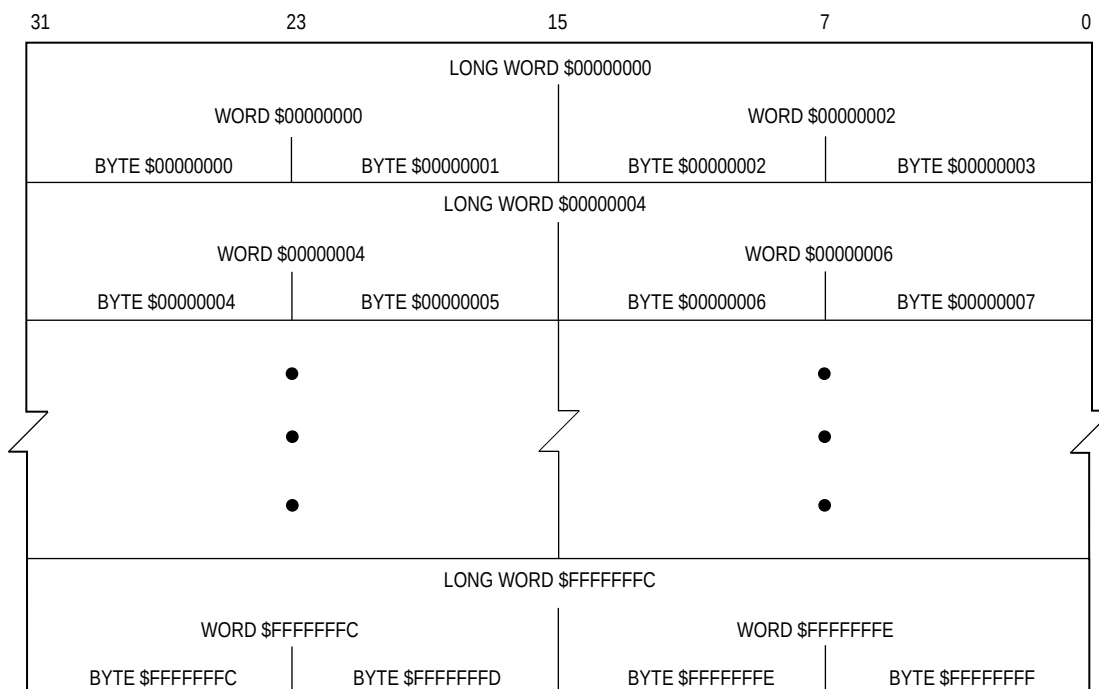


Figure 1-20. Memory Operand Addressing

Figure 1-21 illustrates the organization of IU data formats in memory. A base address that selects one byte in memory, the base byte, specifies a bit number that selects one bit, the bit operand, in the base byte. The MSB of the byte is seven.

The following conditions specify a bit field operand:

1. A base address that selects one byte in memory.
2. A bit field offset that shows the leftmost (base) bit of the bit field in relation to the MSB of the base byte.
3. A bit field width that determines how many bits to the right of the base bit are in the bit field.

The MSB of the base byte is bit field offset 0; the LSB of the base byte is bit field offset 7; and the LSB of the previous byte in memory is bit field offset – 1. Bit field offsets may have values between 2 – 31 to 231 – 1, and bit field widths may range from 1 to 32 bits.

A 16-byte block operand, supported by the MOVE16 instruction, has a block of 16 bytes, aligned to a 16-byte boundary. An address that can point to any byte in the block specifies this operand.

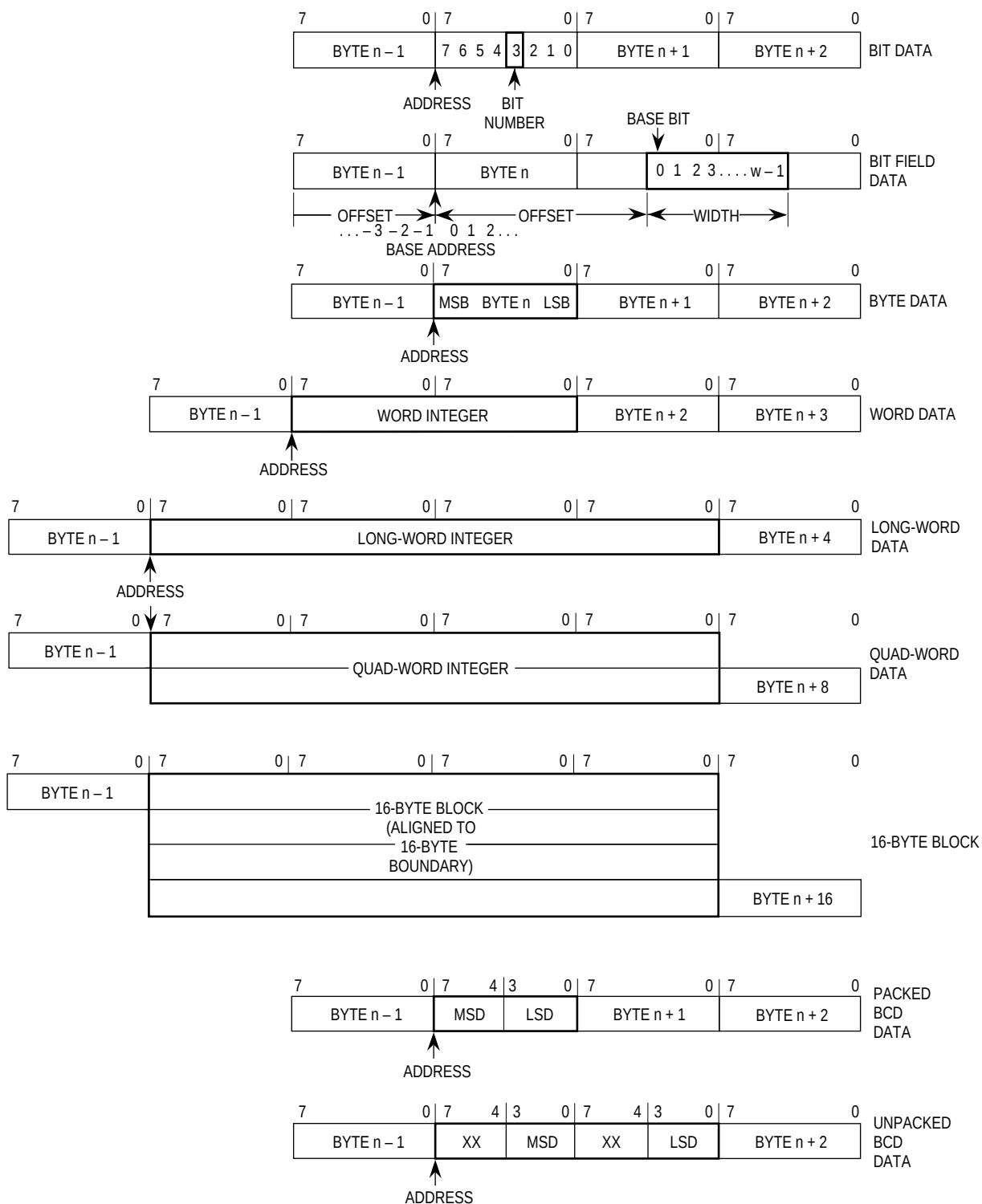


Figure 1-21. Memory Organization for Integer Operands

1.7.3 Organization of Fpu Data Formats in Registers and Memory

The eight, 80-bit floating-point data registers are analogous to the integer data registers and are completely general purpose (i.e., any instruction may use any register). The MC68040 supports only some data formats and types in hardware. Table 1-8 lists the data formats supported by the MC68040.

Table 1-8. MC68040 FPU Data Formats and Data Types

Number Types	Data Formats						
	Single-Precision Real	Double-Precision Real	Extended-Precision Real	Packed-Decimal Real	Byte Integer	Word Integer	Long-Word Integer
Normalized	*	*	*	†	*	*	*
Zero	*	*	*	†	*	*	*
Infinity	*	*	*	†			
NAN	*	*	*	†			
Denormalized	†	†	†	†			
Unnormalized			†	†			

NOTES:
 * = Data Format/Type Supported by On-Chip MC68040 FPU Hardware
 † = Data Format/Type Supported by Software (MC68040FPSP)

Figure 1-22 illustrates the floating-point data format for the single-, double-, and extended-precision binary real data organization in memory.

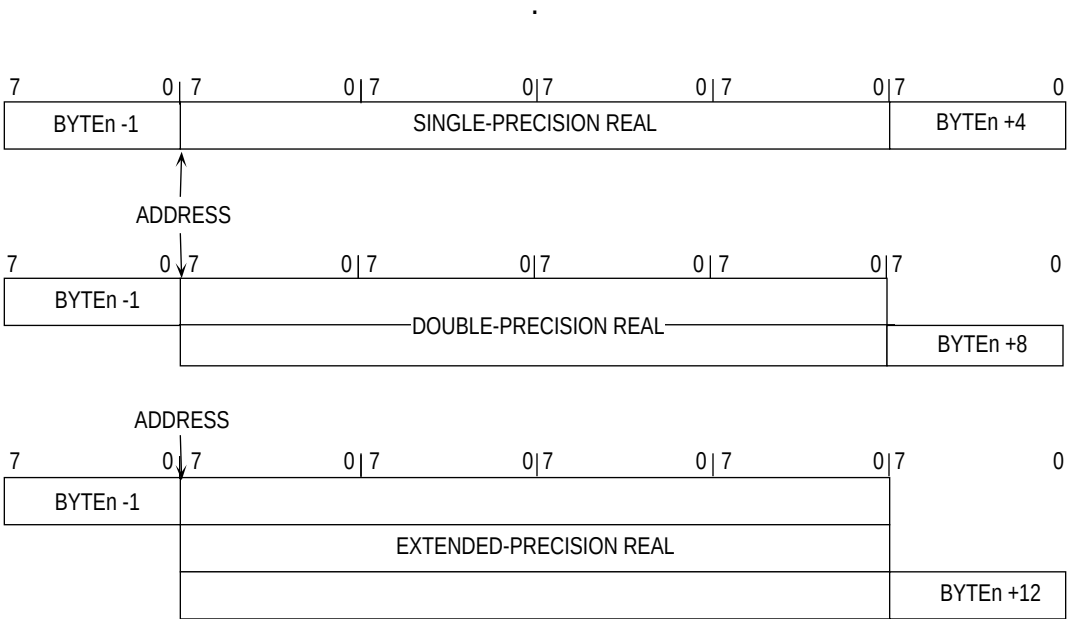


Figure 1-22. Organization of FPU Data Formats in Memory